

Углубленное машинное обучение и искусственный интеллект в Python

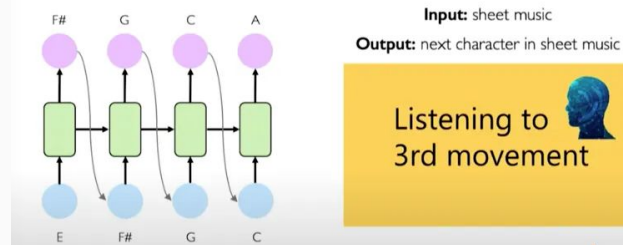
Патласов Дмитрий Александрович, ПГНИУ
dmitriypatlasov@gmail.com

Области применения и ограничения RNN

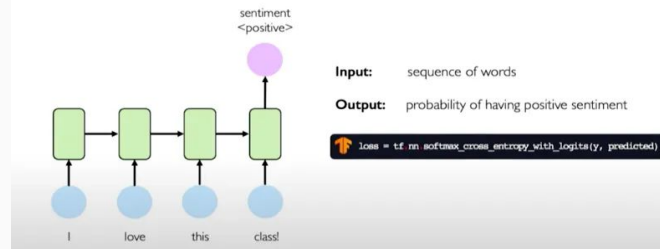
Мы можем обучить модель предсказывать следующую музыкальную ноту в последовательности и генерировать совершенно новые музыкальные последовательности. Мы можем решить проблему прогнозирования следующего временного шага.

Другой пример задачи: классификация по тональности, в которой мы прогнозируем один результат, например, тональность предложения.

Example Task: Music Generation



Example Task: Sentiment Classification



Ограничение повторяющихся моделей

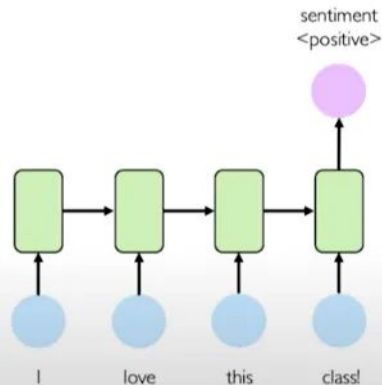
У RNN есть узкое место при кодировании, потому что нам нужно взять много контента и сжать его до представления, которое можно предсказать. Информация может быть потеряна в процессе кодирования.

Запуски вообще неэффективны! Они запрашивают информацию для последовательной обработки. Это делает их очень неэффективными на современном графическом оборудовании.

Рекуррентные модели не обладают большим объемом памяти и не справляются с очень длинными последовательностями, скажем, размером 1000 или 10000.

Согласно [Alayrac et al. (2022)] (<https://paperswithcode.com/paper/flamingo-a-visual-language-model-for-few-shot>), более современные архитектуры, такие как Transformers, "улучшают моделирование долгосрочных зависимостей по сравнению с подходами, основанными на RNN, при этом значительно увеличивая пропускную способность моделей и, следовательно, объем данных, просматриваемых во время обучения".

Limitations of Recurrent Models



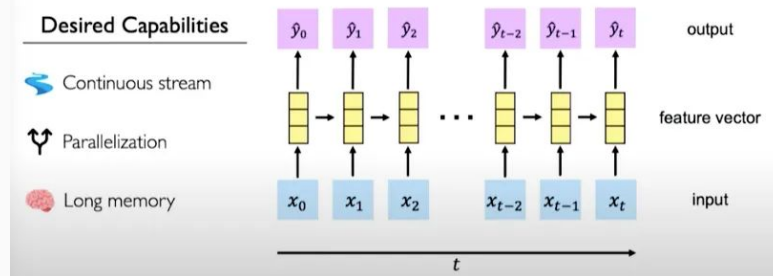
Limitations of RNNs

-  Encoding bottleneck
-  Slow, no parallelization
-  Not long memory

Желаемые возможности были бы следующими:

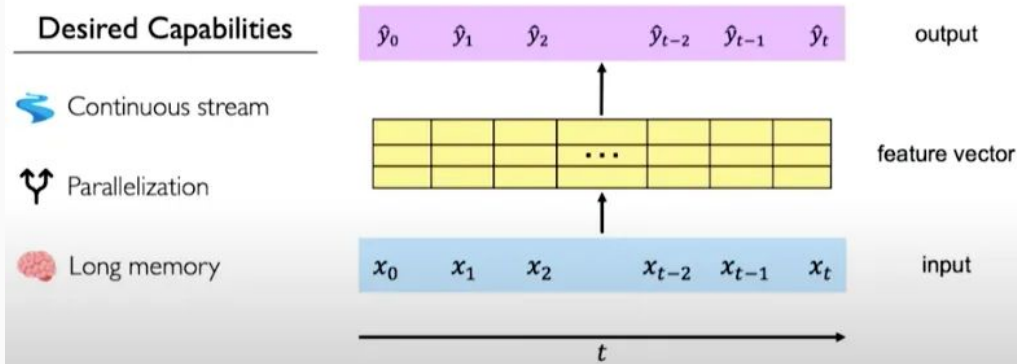
- Непрерывный поток информации, позволяющий преодолеть узкое место в кодировании
- Модель должна быть распараллеливаемой
- Должна быть возможность масштабирования и поддержки большой памяти

Goal of Sequence Modeling



Goal of Sequence Modeling

Can we eliminate the need for recurrence entirely?



Однако этот подход ограничивает нас следующим образом:

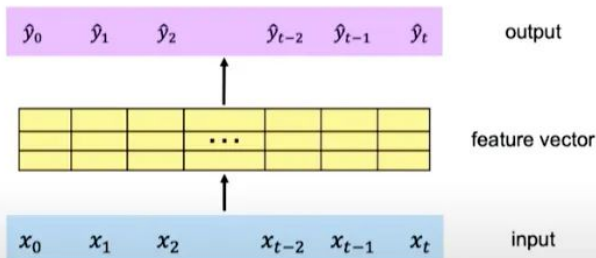
Другая проблема заключается в том, что мы не имеем представления о том, какие моменты в нашей последовательности важны. Это ключевая идея концепции под названием "внимание", которая будет обсуждаться далее.

Goal of Sequence Modeling

Idea 1: Feed everything into dense network

- ✓ No recurrence
- ✗ Not scalable
- ✗ No order
- ✗ No long memory

Can we eliminate the need for recurrence entirely?



Goal of Sequence Modeling

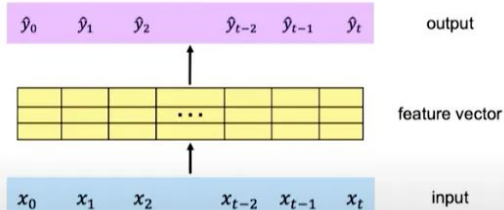
Idea 1: Feed everything into dense network

- ✓ No recurrence
- ✗ Not scalable
- ✗ No order
- ✗ No long memory



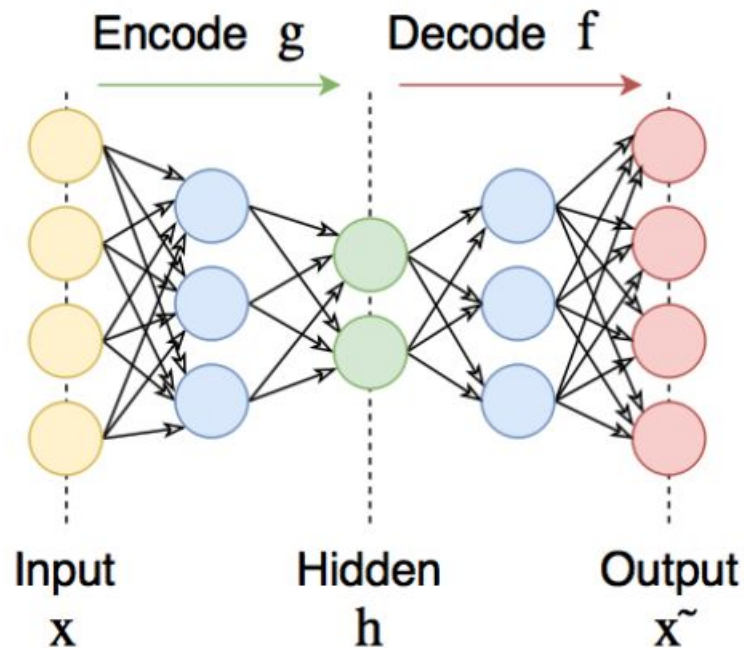
Idea: Identify and attend to what's important

Can we eliminate the need for recurrence entirely?



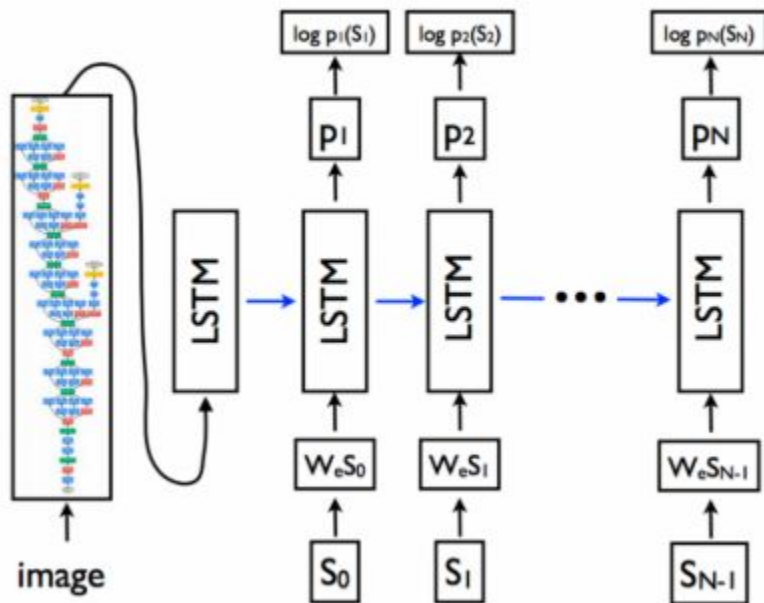
Мультимодальные модели

Автокодировщики

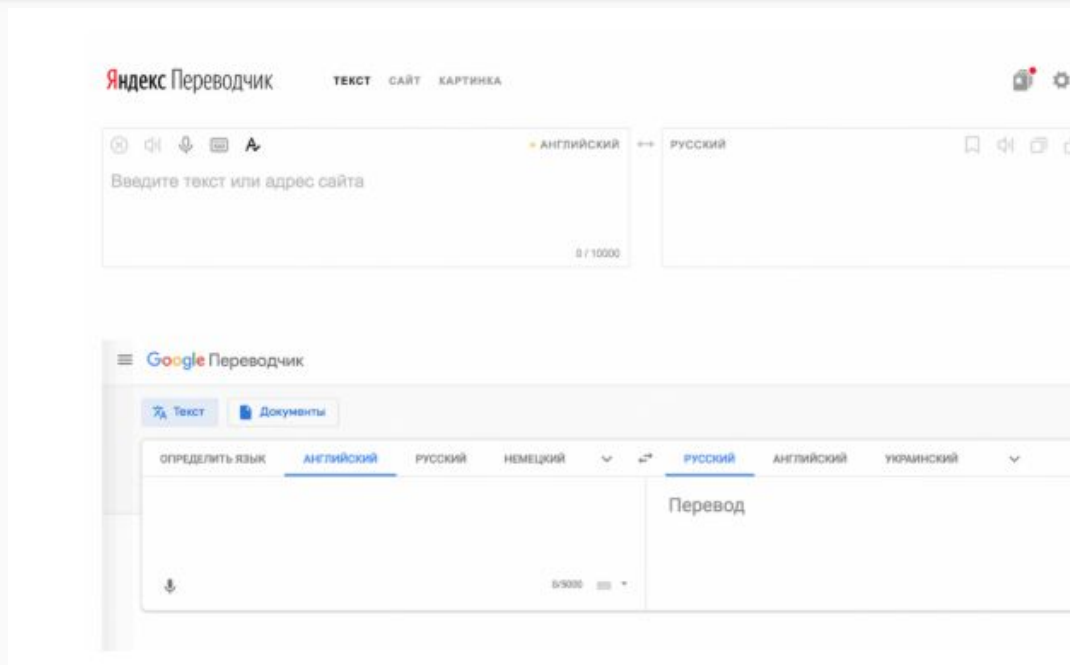


$$\frac{1}{n} \sum_{i=1}^n L(x_i, f(g(x_i))) \rightarrow \min$$

Генерация текста по картинке (2015)



Переводчики



Нейросетевые переводы

- Энкодер пытается понять смысл и закодировать текст
- Декодер пытается раскодировать текст и сгенерировать таргет

Последовательность -> Последовательность

Машинный перевод

Пример:

ITA: Il gatto si e' seduto sul tappetino.



EN: The cat sat on the mat.

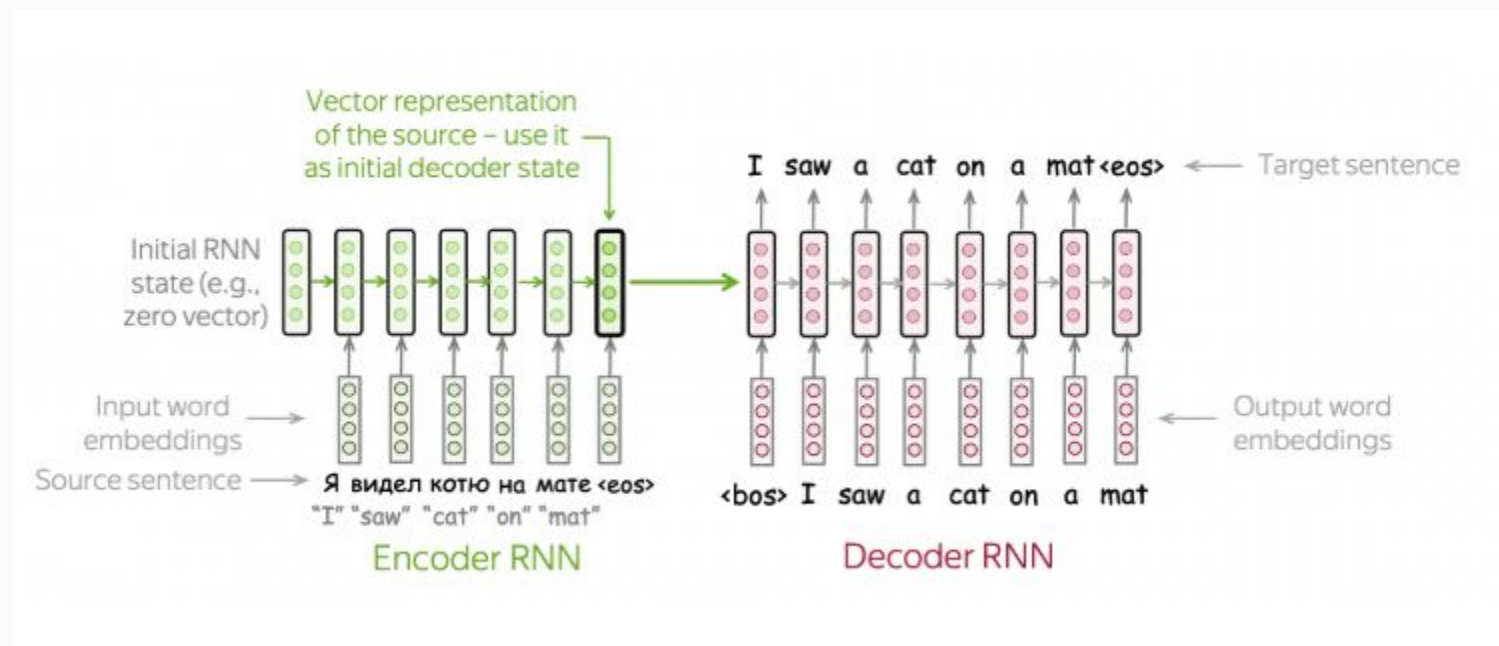
Подход:

seq2seq: имеем один RNN для кодирования исходного предложения, а другой RNN - для предсказания целевого предложения.

В обычной seq2seq для сети decoder мы передаем последнее состояние encoder, но оно не меняется (уже конечное после всех слов) при генерации новых слов с помощью decoder.

Поэтому применим механизм внешнего внимания. Целевой RNN учится (мягко) выравнять предложение с помощью механизма внимания (обращать внимание на разные слова в зависимости от текста, который уже сгенерировали).

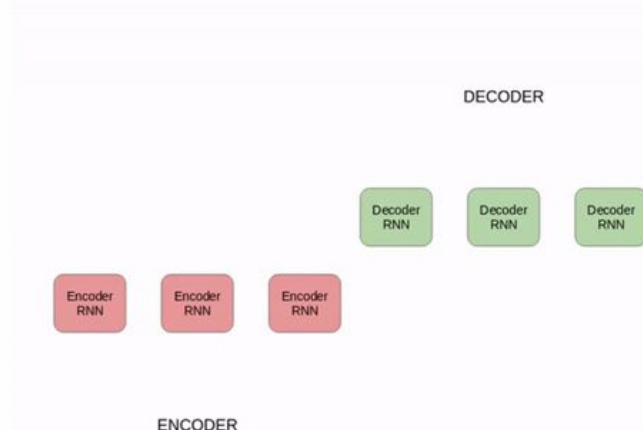
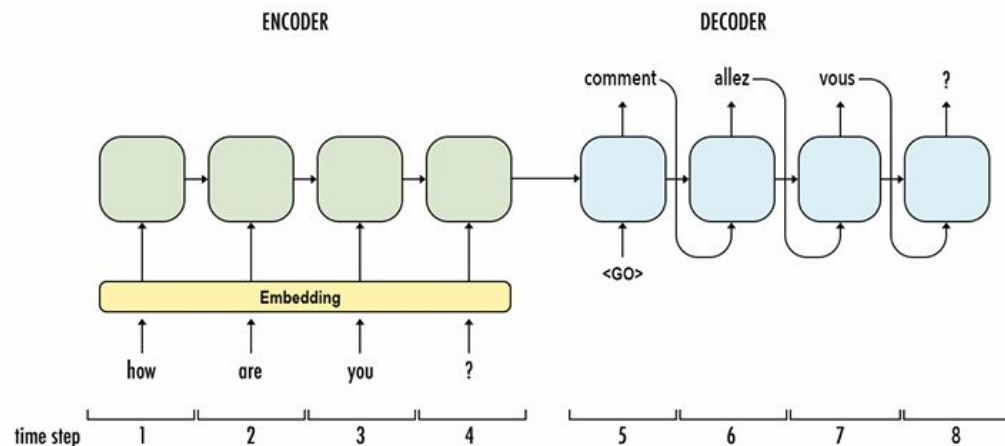
Простейшая модель: RNN энкодер-декодер



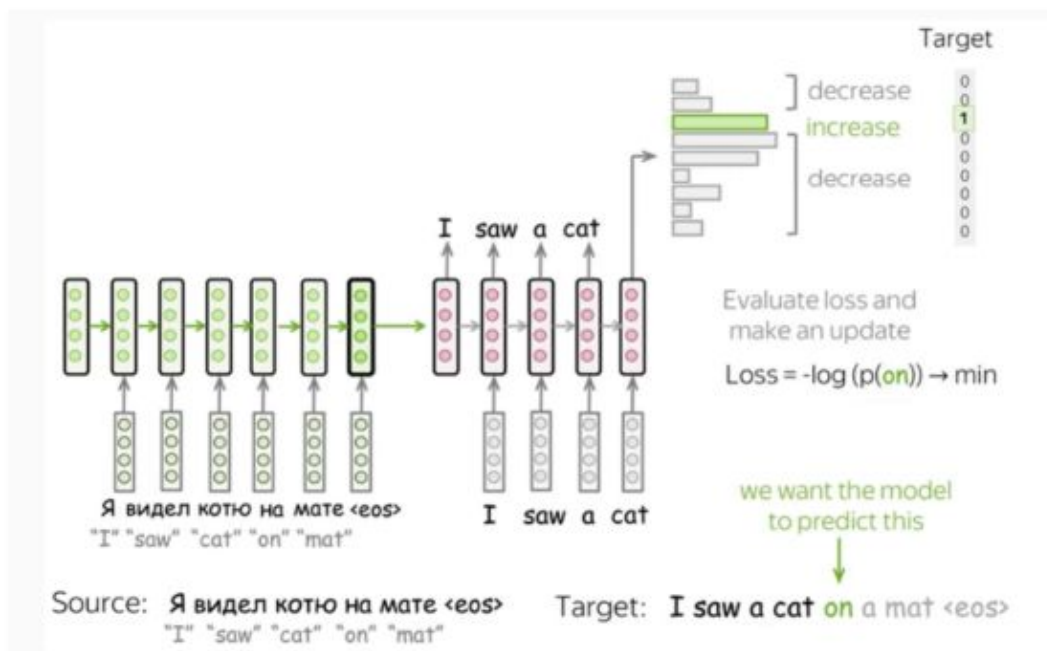
Seq2seq

- Для машинного перевода
- Состоит из двух частей, например две LSTM
- Можно добавить внешнее внимание

Neural Machine Translation SEQUENCE TO SEQUENCE MODEL WITH ATTENTION



Как обучить модель?



Как сделать прогноз?

- **Жадно (greedy decoding)**: на каждом шаге берём токен с самой большой вероятностью

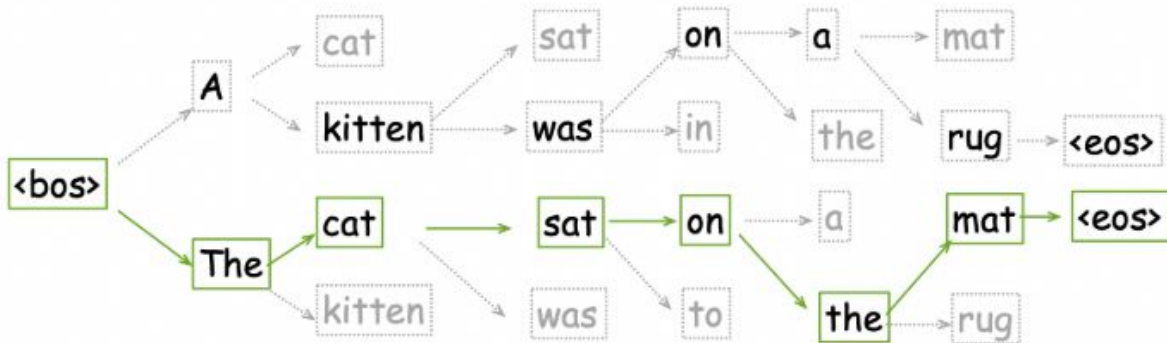
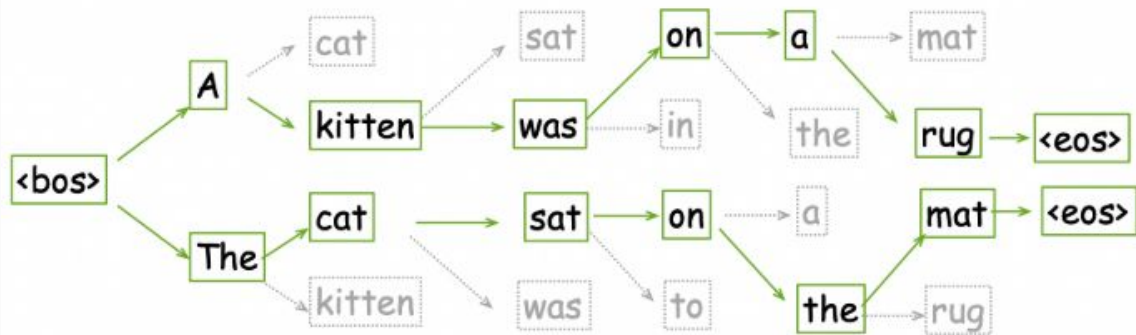
$$\prod_{t=1}^T \arg \max_{y_t} p(y_t \mid y_{<t}, x) \neq \arg \max_y \prod_{t=1}^T p(y_t \mid y_{<t}, x)$$

- Наш прогноз это последовательность, жадный способ на каждом шаге выбирает локальный оптимум
- Перебирать все траектории, чтобы найти глобальный оптимум очень дорого

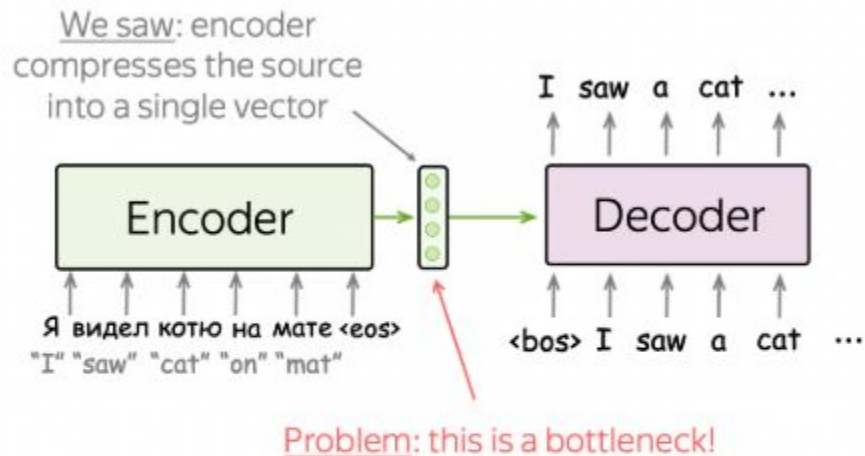
Beam Search

- Давайте поддерживать несколько самых вероятных траекторий
- Такая стратегия называется **Beam Search**
- Число траекторий, которое мы помним будет гиперпараметром (больше 10 брать не стоит)

Beam Search

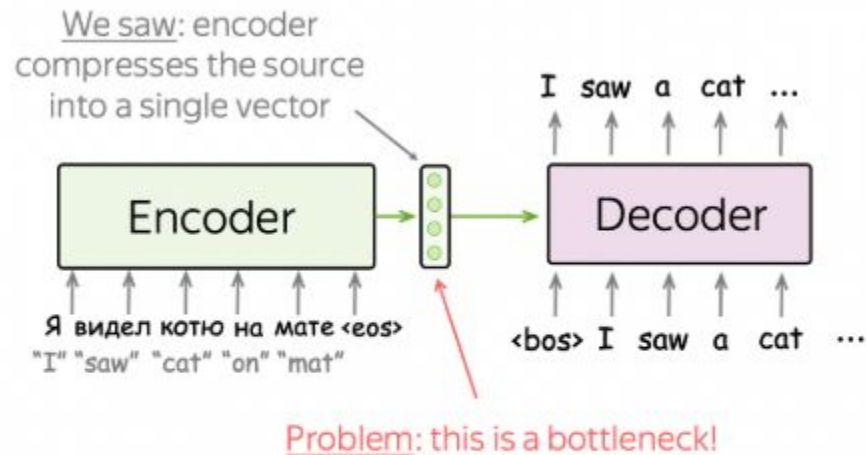


В чём проблема простой RNN модели?



- Сжать предложение в вектор сложно
- Декодеру может понадобиться информация о разных частях предложения для его расшифровки

В чём проблема простой RNN модели?



- **Attention**: на разных шагах надо позволить модели фокусироваться на разных входных токенах

Проблемы seq2seq-архитектуры

- Нужно сжать весь текст в один вектор
- Теряется информация о первых словах
- Декодер тоже может терять информацию по мере генерации последовательности
- Может, нам поможет BiLSTM?

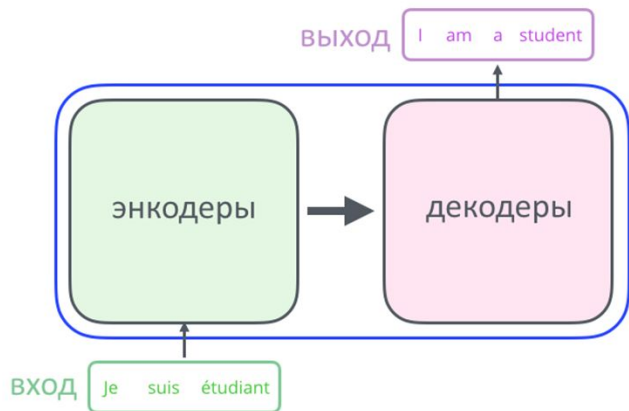
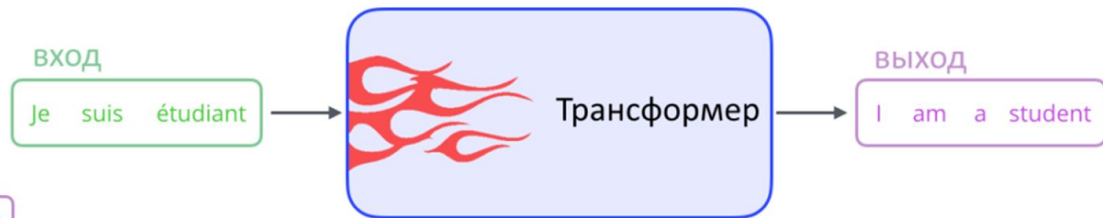
Проблемы seq2seq-архитектуры

- Нужно сжать весь текст в один вектор
 - Теряется информация о первых словах
 - Декодер тоже может терять информацию по мере генерации последовательности
-
- Можно использовать BiLSTM, но тогда будет теряться информация о словах в середине
 - И непонятно, как им декодировать

Attention is All You Need!

Трансформеры (transformers)

Как устроен трансформер



Attention is All You Need! (2017)

- RNN это очень долго! Всегда, чтобы найти следующий токен, надо знать предыдущий
- Backward pass идёт ещё и через время :(
- Transformer — нейросетевая архитектура для задач seq2seq, основанная исключительно на полносвязных слоях
- Превзошла существовавшие seq2seq архитектуры как по качеству, так и по скорости работы
- Основной элемент — multi-head self-attention

Зачем нам внимание?

Для начала вспомним, что основным подходом для работы с последовательностями до 2017 года (выхода оригинальной статьи про Transformer) являлось использование рекуррентных нейронных сетей, или RNN. Однако у такого подхода есть несколько известных минусов:

- Во-первых, RNN содержат всю информацию о последовательности в скрытом состоянии, которое обновляется с каждым шагом. Если модели необходимо «вспомнить» что-то, что было сотни шагов назад, то эту информацию необходимо хранить внутри скрытого состояния и не заменять чем-то новым. Следовательно, придется иметь либо очень большое скрытое состояние, либо мириться с потерей информации.
- Во-вторых, обучение рекуррентных сетей сложно распараллелить: чтобы получить скрытое состояние RNN-слоя для шага $i + 1$, вам необходимо вычислить состояние для шага i . Таким образом, обработка батча примеров длиной 1000 должна потребовать 1000 последовательных операций, что занимает много времени и не очень эффективно работает на GPU, созданных для параллельных вычислений.

Зачем нам внимание?

Обе этих проблемы затрудняют применение RNN к настоящему длинным последовательностям: даже если вы дождетесь конца обучения, ваша модель по своей конструкции будет так или иначе терять информацию о том, что было в начале текста. Хочется иметь способ «читать» последовательность так, чтобы в каждый момент времени можно было обратиться к произвольному моменту из прошлого за константное время и без потерь информации.

Таким способом и является лежащий в основе Transformer механизм self-attention, о котором далее пойдет речь. Как мы узнаем позже, благодаря своей универсальности и масштабируемости этот механизм оказался применим к множеству задач помимо обработки естественного языка.

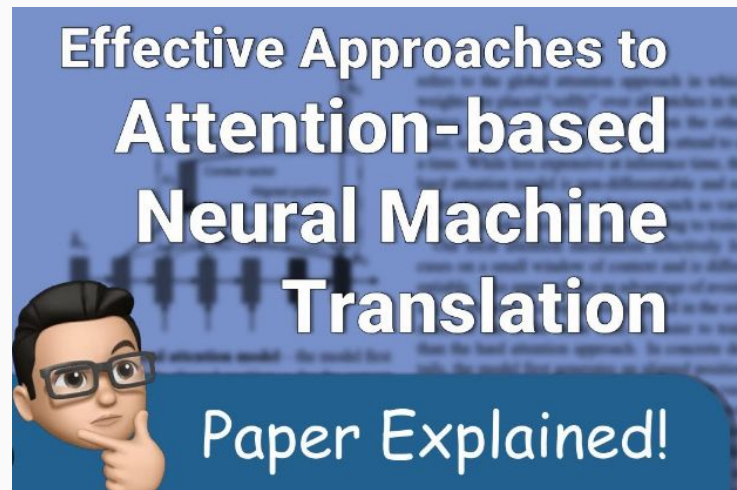
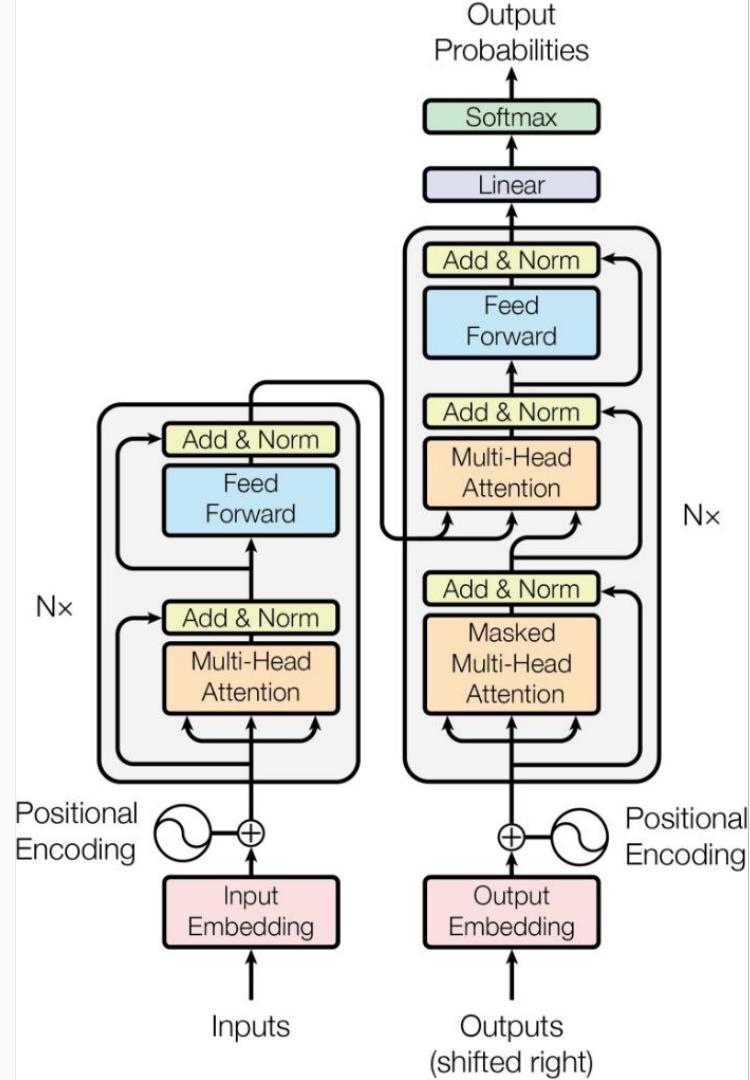
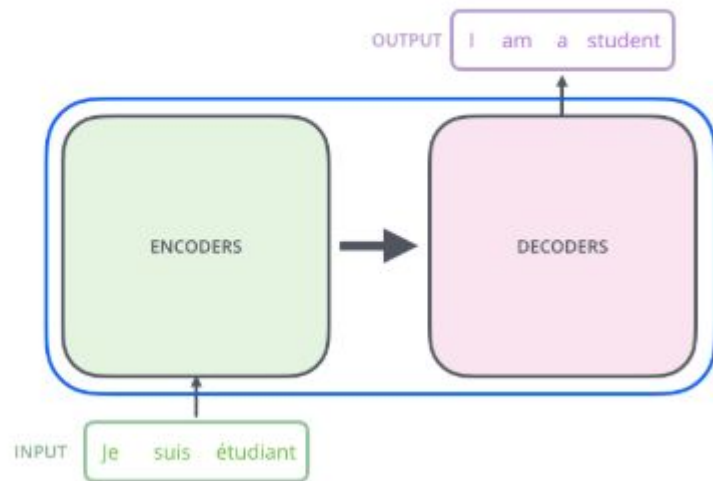


Схема трансформера



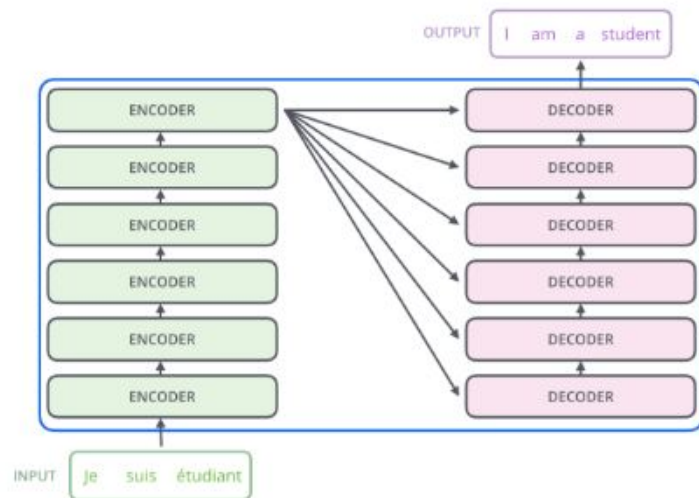
Transformer

Верхнеуровнего - это просто энкодер и декодер



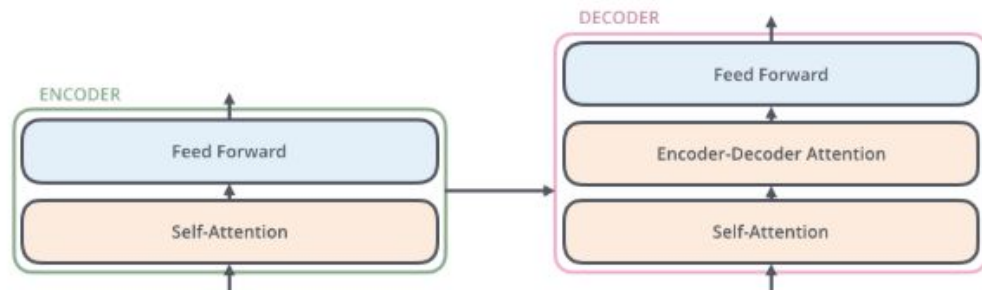
Transformer

Энкодер и декодер состоят из одинаковых блоков; веса во всех блоках разные



Transformer

В энкодере происходят две вещи: сначала вход прогоняется через self-attention, а затем — через полносвязный слой. В декодере помимо обычного self-attention есть ещё и attention из энкодера.



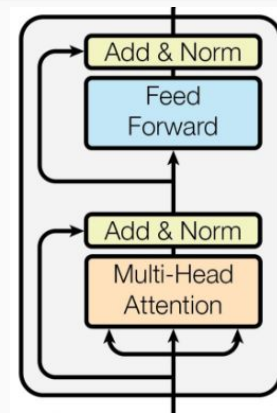
Энкодер

Он по очереди применяет к исходной последовательности N блоков:

Каждый блок выдаёт последовательность такой же длины. В нём есть два важных слоя, multi-head attention и feed-forward. После каждого из них к выходу прибавляется вход (это стандартный подход под названием residual connection) и затем активации проходят через слой layer normalization: на рисунке эта часть обозначена как «Add & Norm».

У декодера схема похожая, но внутри каждого из N блоков два слоя multi-head attention, в одном из которых используются выходы энкодера.

Давайте подробнее обсудим каждую из составляющих частей этого механизма.



Слой внимания

Первая часть transformer-блока — это слой self-attention. От обычного внимания его отличает то, что выходом являются новые представления для элементов той же последовательности, что мы подали на вход, причем каждый элемент этой последовательности напрямую взаимодействует с каждым.

Если говорить более подробно, то в вычислении внимания для последовательности будет участвовать три обучаемых матрицы W_Q , W_K , W_V . Представление x_i каждого элемента входной последовательности мы умножаем на W_Q , W_K , W_V , получая вектор-строки q_i, k_i, v_i (i – номер элемента), которые соответственно называются *запросами*, *ключами* и *значениями* (query, key и value). Их роли можно условно описать следующим образом:

- q_i – запрос к базе данных;
- k_i – ключи хранящихся в базе значений, по которым будет осуществляться поиск;
- v_i – сами значения.

Слой внимания

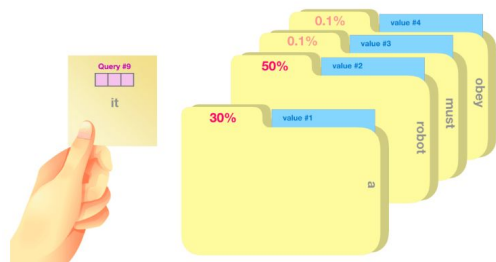
Близость запроса к ключу можно определять, например, с помощью скалярного произведения:

$$\text{self-attention_weights}_i = \text{softmax} \left(\frac{q_i k_1^T}{C}, \frac{q_i k_2^T}{C}, \dots \right),$$

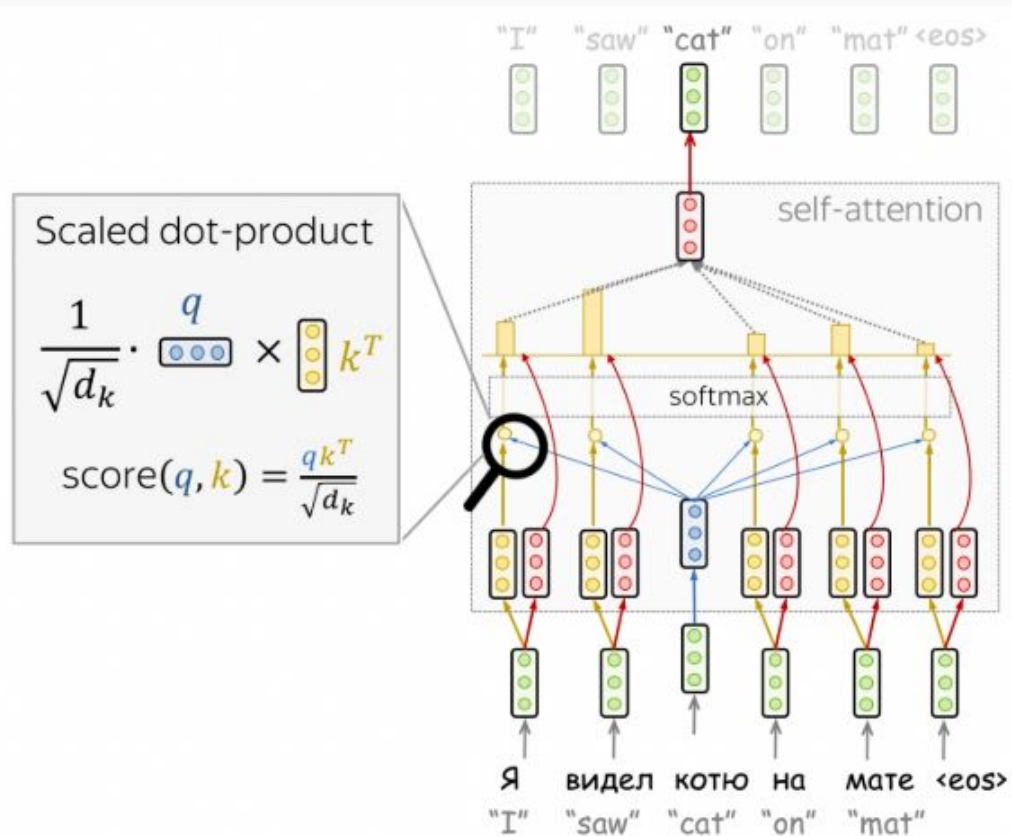
где C – некоторая нормировочная константа. Именно так и делали в исходной статье; в качестве нормировочной константы брался корень $\sqrt{d_k}$ из размерности ключей и значений. Теперь мы складываем значения v_i с полученными коэффициентами. Это и будет выходом слоя self-attention. В векторизованном виде можно записать:

$$\text{self-attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V,$$

где Q, K, V – матрицы запросов, ключей и значений соответственно, в которых по строкам записаны q_i, k_i, v_i , а softmax берётся построчно.



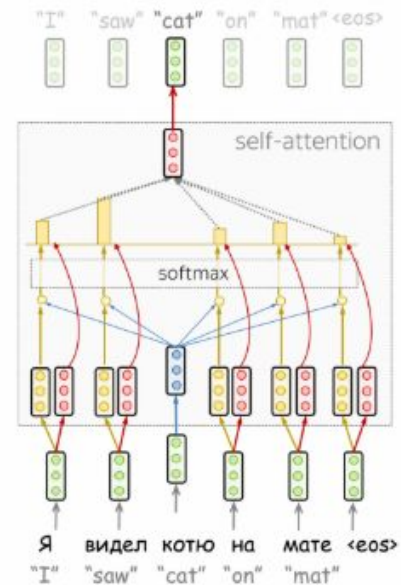
Query, Key, Value



Query, Key, Value

$$\text{Attention}(q, k, v) = \overbrace{\text{softmax}\left(\frac{qk^T}{\sqrt{d_k}}\right)}^{\text{Attention weights}} v$$

from to
vector dimensionality of K, V



Внутреннее внимание

Перемножая x_1 на матрицу весов W^Q получаем q_1 , вектор "query", связанный с этим словом.

В итоге мы создаем проекцию "query", "key" и "value" каждого слова во входном предложении.

Input

Thinking

Machines

Embedding

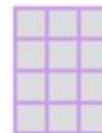
x_1 

x_2 

Queries

q_1 

q_2 



W^Q

Keys

k_1 

k_2 



W^K

Values

v_1 

v_2 



W^V

Query, Key, Value*

- Каждая строка в матрице **X** соответствует слову во входном предложении
- Мы определяем какой в итоге результат будет для данного слова: это его вклад и вклад других слов в предложении

Input

Embedding

Queries

Keys

Values

Score

Divide by 8 ($\sqrt{d_k}$)

Softmax

Softmax
X
Value

Sum

Thinking

x_1

q_1

k_1

v_1

$q_1 \cdot k_1 = 112$

14

0.88

v_1

z_1

Machines

x_2

q_2

k_2

v_2

$q_2 \cdot k_2 = 96$

12

0.12

v_2

z_2

Несколько слов *

Каждая строка в матрице **X** соответствует слову во входном предложении

А количество строк – это размер эмбединга для каждого слова

Запишем в **матричной форме**:

$$\text{softmax}\left(\frac{\begin{matrix} \text{Q} \\ \text{2x3 grid} \end{matrix} \times \begin{matrix} \text{K}^T \\ \text{3x2 grid} \end{matrix}}{\sqrt{d_k}}\right) \begin{matrix} \text{V} \\ \text{2x2 grid} \end{matrix}$$
$$= \begin{matrix} \text{Z} \\ \text{2x2 grid} \end{matrix}$$

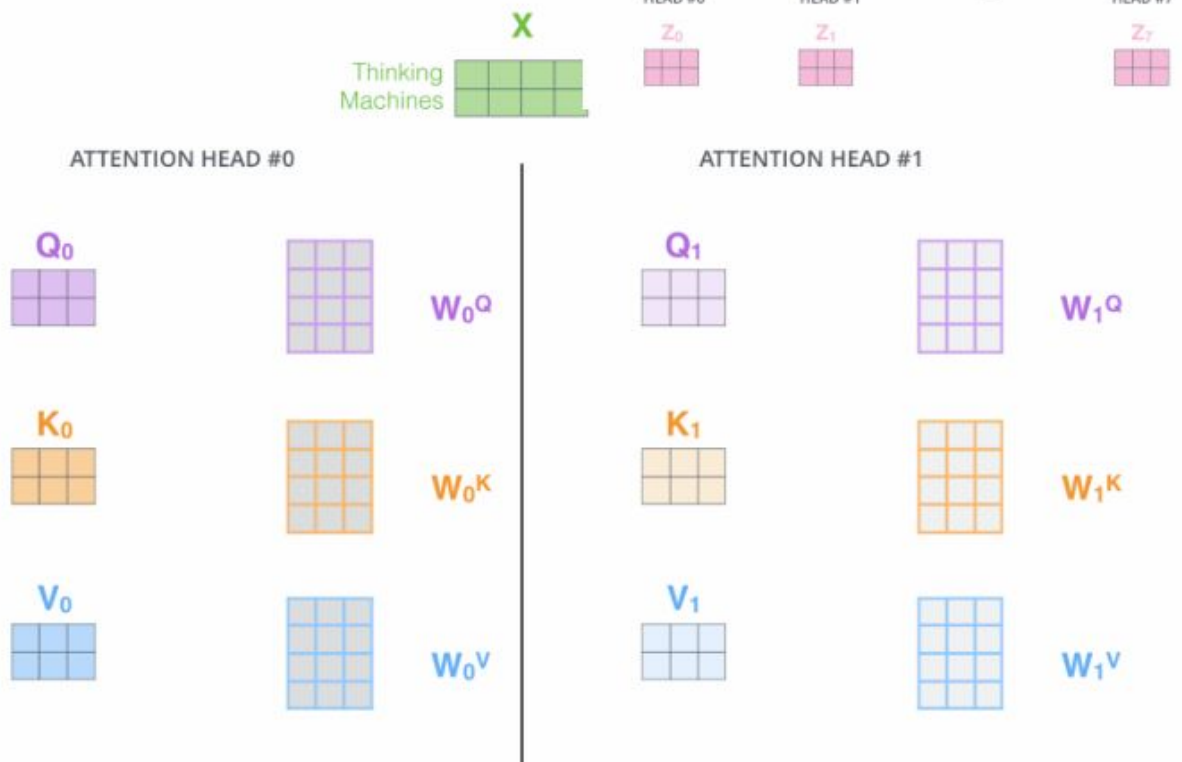
$$\begin{matrix} \text{X} \\ \text{2x4 grid} \end{matrix} \times \begin{matrix} \text{W}^Q \\ \text{4x3 grid} \end{matrix} = \begin{matrix} \text{Q} \\ \text{2x3 grid} \end{matrix}$$

$$\begin{matrix} \text{X} \\ \text{2x4 grid} \end{matrix} \times \begin{matrix} \text{W}^K \\ \text{4x3 grid} \end{matrix} = \begin{matrix} \text{K} \\ \text{2x3 grid} \end{matrix}$$

$$\begin{matrix} \text{X} \\ \text{2x4 grid} \end{matrix} \times \begin{matrix} \text{W}^V \\ \text{4x2 grid} \end{matrix} = \begin{matrix} \text{V} \\ \text{2x2 grid} \end{matrix}$$

Attention heads *

- При многоголовом внимании мы обрабатываем отдельные матрицы весов Q/K/V для каждой головы
- Это приводит к разным матрицам Q/K/V
- Как и раньше, мы умножаем X на матрицы $WQ/WK/WV$, чтобы получить матрицы Q/K/V



Attention result

- Когда мы получили результат для каждой головы нам нужно его объединить
- Используем еще одну матрицу W (которую тоже обучаем)
- Это пример простой линейной регрессии

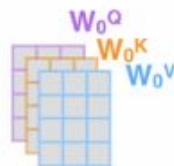
1) This is our input sentence*

Thinking
Machines

2) We embed each word*



3) Split into 8 heads. We multiply X or R with weight matrices



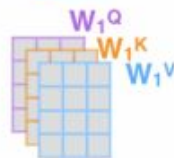
4) Calculate attention using the resulting $Q/K/V$ matrices



5) Concatenate the resulting Z matrices, then multiply with weight matrix W^O to produce the output of the layer



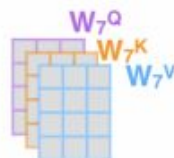
* In all encoders other than #0, we don't need embedding. We start directly with the output of the encoder right below this one



...

...

...



W^O



🔥 КАК РАБОТАЕТ ВНИМАНИЕ (ATTENTION) В ТРАНСФОРМЕРАХ

Механизм внимания - это способ для модели "смотреть" на разные слова в тексте и решать, какие из них сейчас важнее.

Представь фразу: "Я положил ключи на стол, потому что он был рядом".

Слово "он" должно ссылаться на "стол", а не на "ключи".

Внимание делает ровно это - для каждого слова вычисляет, на какие другие слова нужно опираться сильнее.

Как это работает в одном абзаце:

Для каждого токена модель строит три вектора - Query (что я ищу), Key (что я из себя представляю), Value (какую информацию несусь).

Считается "похожесть" Query к каждому Key - это оценки важности.

Оценки нормализуются (softmax), чтобы получились веса от 0 до 1.

Итоговое представление токена - это взвешенная сумма Value по всем токенам.

То есть модель смешивает информацию из контекста, но делает это умно - больше веса тем словам, которые сейчас важны.

Почему это суперсила:

вместо того чтобы читать текст строго слева направо, модель может связывать дальние куски, находить нужные факты и строить смысловые зависимости. Именно поэтому трансформеры так хорошо работают с языком, кодом и длинным контекстом.

```
import numpy as np
```

```
def softmax(x: np.ndarray) -> np.ndarray:
    x = x - np.max(x) # стабилизация
    e = np.exp(x)
    return e / np.sum(e)
```

```
# Допустим, у нас 3 токена в контексте.
# Для простоты возьмём маленькую размерность векторов (d=2).
# Query - "что я ищу"
# Key - "что я такое"
# Value - "какую инфу несусь"
```

```
Q = np.array([1.0, 0.5]) # Query для текущего токена (например, слово "он")
```

```
K = np.array([
    [1.0, 0.0], # Key токена 1 (например, "ключи")
    [0.9, 0.1], # Key токена 2 (например, "стол")
    [0.0, 1.0], # Key токена 3 (например, "рядом")
])
```

```
V = np.array([
    [10.0, 0.0], # Value токена 1
    [0.0, 10.0], # Value токена 2
    [5.0, 5.0], # Value токена 3
])
```

```
# 1) Считаем "похожесть" Q на каждый K через скалярное произведение
scores = K @ Q # shape: (3,)
print("scores:", scores)
```

```
# 2) Нормализуем оценки -> получаем веса внимания
weights = softmax(scores)
print("attention weights:", weights)
```

```
# 3) Итоговый вектор = взвешенная сумма Value
output = weights @ V # shape: (2,)
print("output (context mixed):", output)
```

```
# Интерпретация:
```

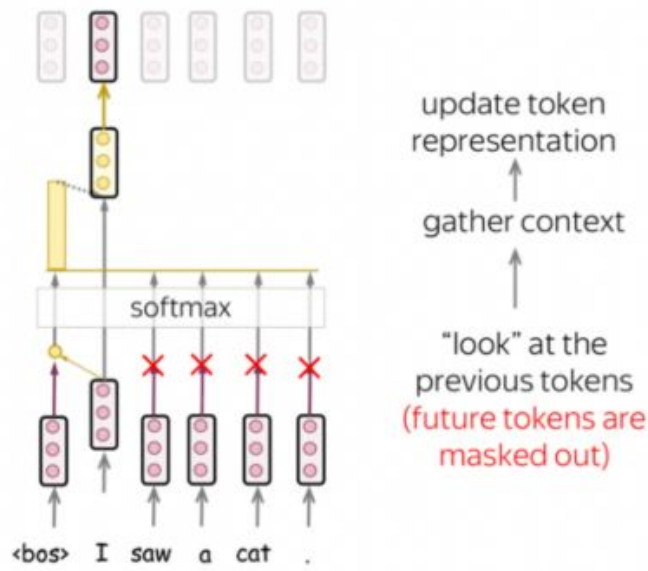
```
# Если вес токена 2 ("стол") самый большой, значит модель "смотрит" на него сильнее всего.
```

```
scores: [1.  0.95 0.5 ]
attention weights: [0.39096708 0.37189939 0.23713352]
output (context mixed): [5.09533845 4.90466155]
```

Masked Self-Attention

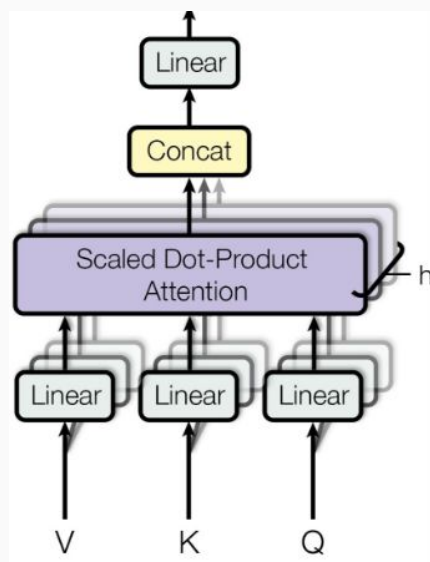
In the decoder, we forbid looking at future tokens – we don't know them

Note: in training, decoder processes all target tokens at once – without masks, it would see future

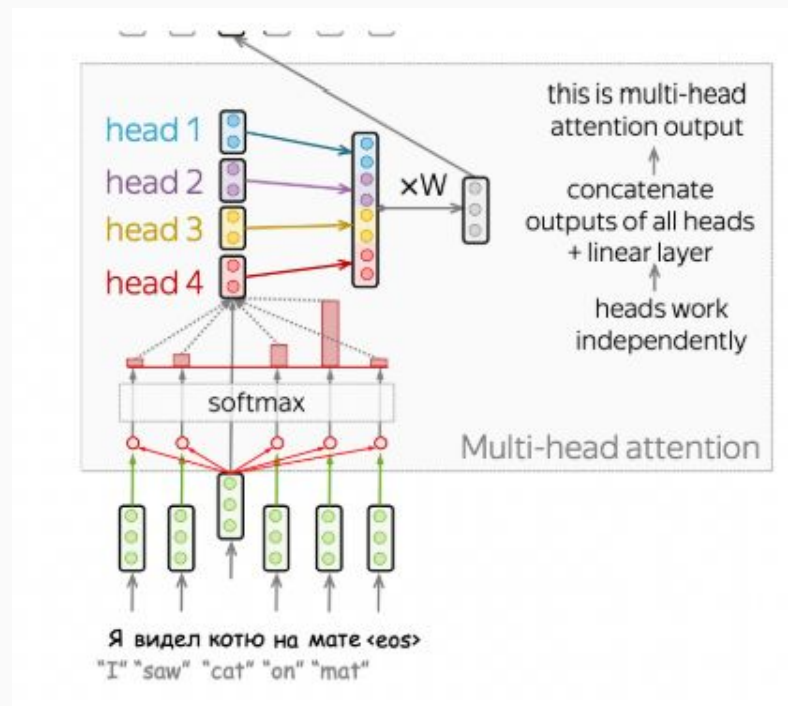


Multi-head attention

Один набор Q , K и V может отражать только один вид зависимостей между токенами, и матрицы извлекают лишь ограниченный набор информации из входных представлений. Чтобы скомпенсировать эту неоптимальность, авторы архитектуры предложили подход с несколькими «головами» внимания (multi-head attention): по сути вместо одного слоя внимания мы применяем несколько параллельных с разными весами, а потом агрегируем результаты. Рисунок ниже показывает, как выглядит multi-head attention:

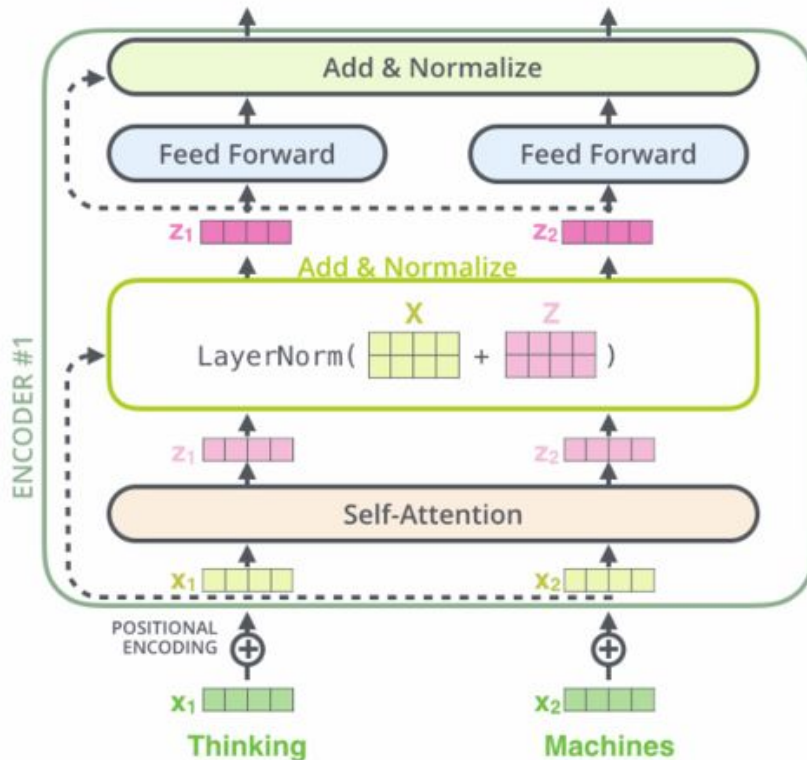


Multi-Head Attention



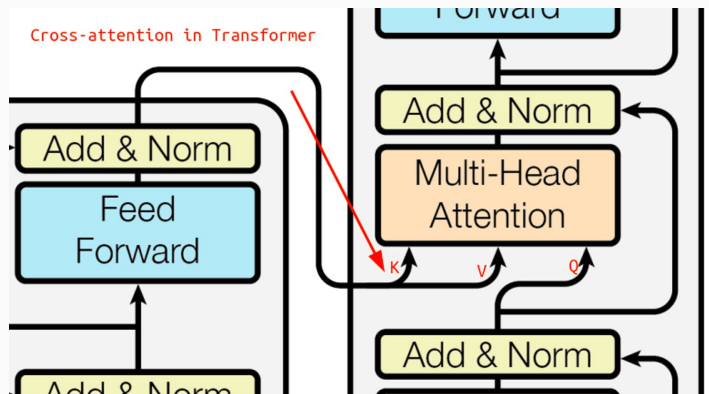
Add and Normalize

- Весь процесс вычислений в одном слое энкодера
- Все входные слова мы можем посчитать вместе и параллельно (как в CNN)
- В RNN мы и входной текст подаем по одному слову (формируем память)
- В трансформере нет скрытой памяти – новый сгенерированный текст это вся наша «память»



Особенности слоя внимания в декодере

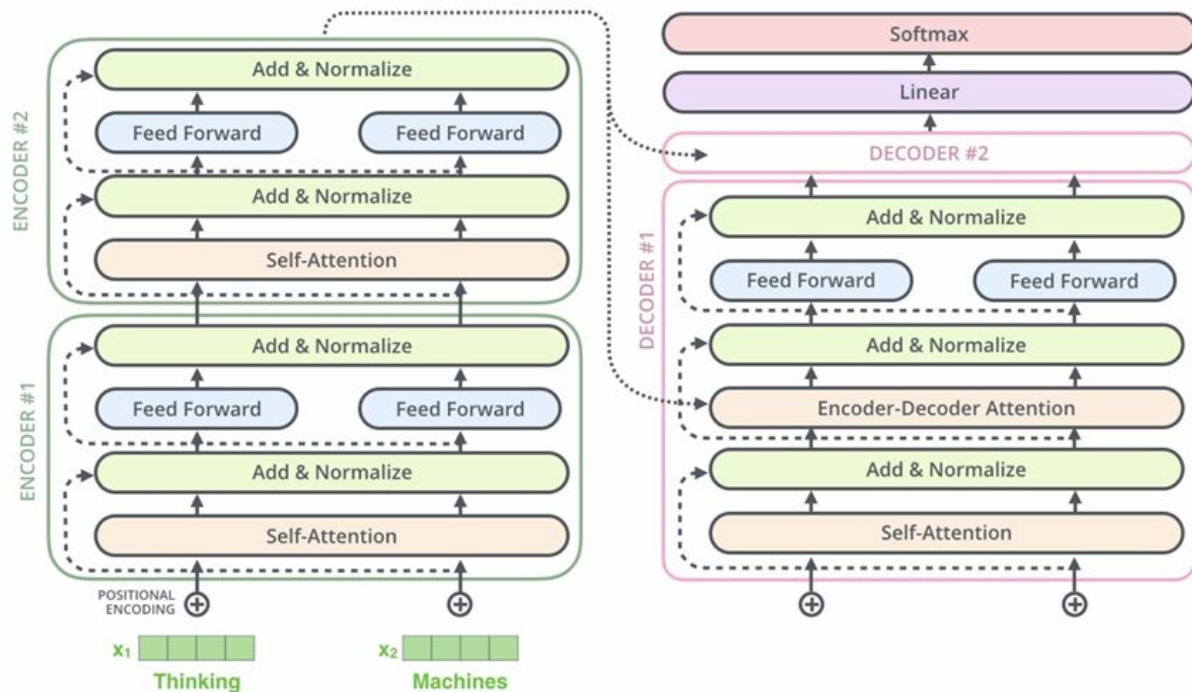
Как мы уже упоминали выше, в декодере один из attention-слоёв является слоем кросс-внимания (cross-attention), в котором запросы берутся из выходной последовательности, а ключи и значения — из входной (то есть из результатов работы энкодера)



Также стоит учитывать, что в описанном выше виде внимания каждый токен будет «смотреть» на всю последовательность, что нежелательно для декодера. Действительно, на этапе генерации мы будем порождать по одному токenu за шаг, и доступ к последующим шагам на этапе обучения приведет к утечке информации в декодере и низкому качеству модели. Чтобы избежать этой проблемы, при обучении к вниманию нужно применять авторегрессивную маску, вручную обращая в $-\infty$ веса до softmax для токенов из будущего, чтобы после softmax их вероятности стали нулевыми.

Encoder-decoder connection *

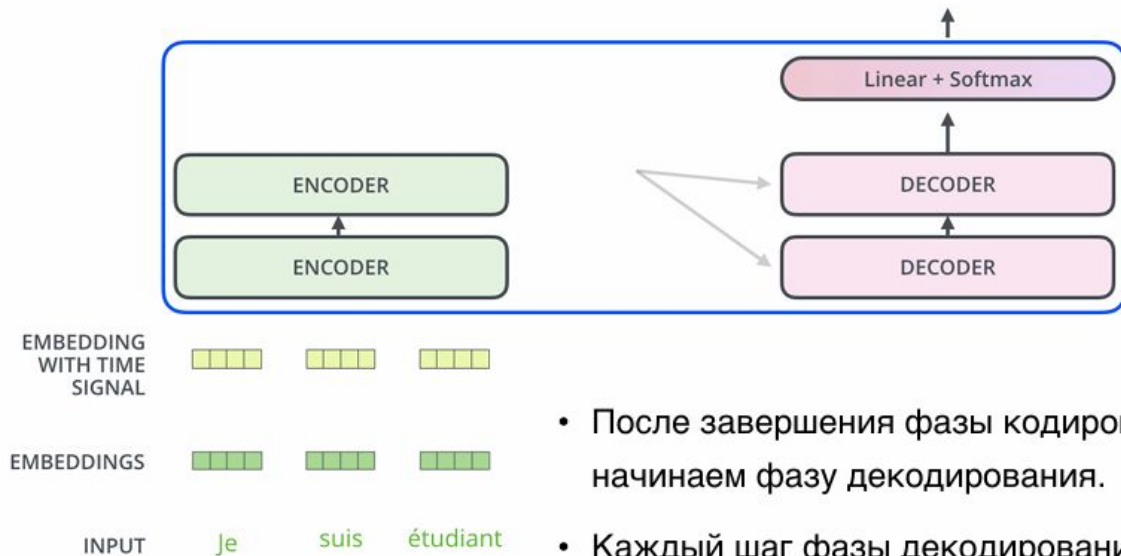
- Вместе два слоя энкодера и два слоя декодера для двух входных слов
- Все входные данные мы уже имеем в тексте (как в авторегрессии)
- Нам НЕ нужно подавать текст только по одному слову как в RNN на вход нейросети
- Но генерация только по одному слову



Decoder

Decoding time step: 1 2 3 4 5 6

OUTPUT



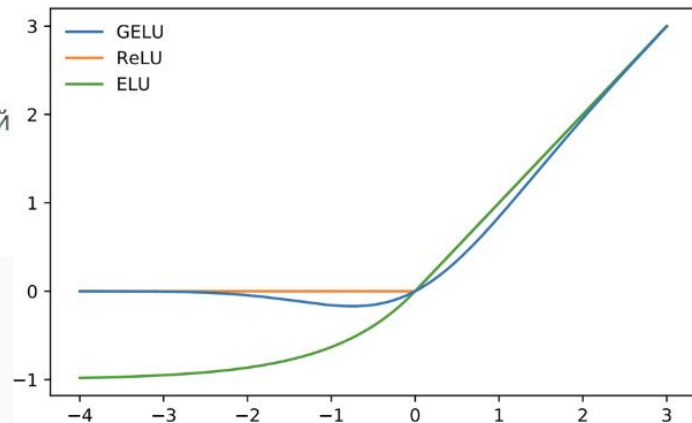
- После завершения фазы кодирования мы начинаем фазу декодирования.
- Каждый шаг фазы декодирования выводит элемент из выходной последовательности (в данном случае предложение перевода на английский язык)

Полносвязный слой и нормализация

Вторая часть трансформерного блока называется *feed-forward network* (FFN) и представляет собой два обычных полносвязных слоя, применяемых независимо к каждому элементу входной последовательности. В последних архитектурах размер промежуточного представления (то есть выхода первого слоя) бывает весьма большим — в 4 раза больше выходов блока. Из-за этого вычислительной стоимостью FFN не стоит пренебрегать: несмотря на квадратичную асимптотику внимания, в больших моделях или на коротких последовательностях FFN может занимать существенно больше времени по сравнению с *self-attention*. В виде формулы применение FFN можно представить так:

$$\text{FFN}(x) = \text{act}(xW_1 + b_1)W_2 + b_2,$$

Промежуточные активации *act* в FFN бывают разными: начиналось всё с широко известной ReLU, но в какой-то момент сообщество перешло на GELU (Gaussian Error Linear Unit), с формулой $x\Phi(x)$, где Φ — функция распределения стандартной нормальной случайной величины.



Кодирование позиций (позиционное кодирование)

Все операции внутри transformer-блока, строго говоря, инвариантны к порядку элементов в последовательности. Например, результат внимания зависит от скалярных произведений между эмбедингами токенов, но расположение этих токенов внутри текста значения не имеет.

Таким образом, итоговые представления каждого токена на выходе из модели будут одинаковыми вне зависимости от порядка слов, что вряд ли нас устроит. Как с этим справиться?

Кодирование позиций (позиционное кодирование)

На помощь приходит такая вещь, как позиционные эмбединги. Это вспомогательные представления, которые прибавляются к обычным эмбедингам токенов входной последовательности и позволяют слоям внимания различать одинаковые токены на разных местах. Исторически первым подходом были фиксированные эмбединги, однозначно кодирующие позицию тригонометрическими функциями (ниже pos — номер позиции, i — индекс элемента в векторе, кодирующем эту позицию, d — размерность эмбединга):

$$\begin{aligned} \text{PE}_{(pos, 2i)} &= \sin\left(\frac{pos}{10000^{2i/d}}\right), \\ \text{PE}_{(pos, 2i+1)} &= \cos\left(\frac{pos}{10000^{2i/d}}\right). \end{aligned}$$

Кодирование позиций (позиционное кодирование)

С момента появления архитектуры Transformer, однако, появилось множество других способов кодировать позиции токенов. Например, можно просто сделать позиционные эмбединги обучаемыми наряду с эмбедингами токенов. Иной подход — напрямую учесть тот факт, что нам важны не абсолютные позиции токенов, а расстояние между ними, и обучать относительные позиционные представления: подобный подход заметно улучшает качество на чувствительных к порядку слов задачах, а его более современные модификации регулярно используются в самых мощных моделях.

The diagram illustrates the construction of relative position representations. It shows two 5x5 matrices being added together, with the result scaled by a factor m .

Matrix 1 (Learned): A 5x5 matrix where each element is a product of a query vector q_i and a key vector k_j . The elements are arranged along the main diagonal and its lower triangle.

$q_1 \cdot k_1$				
$q_2 \cdot k_1$	$q_2 \cdot k_2$			
$q_3 \cdot k_1$	$q_3 \cdot k_2$	$q_3 \cdot k_3$		
$q_4 \cdot k_1$	$q_4 \cdot k_2$	$q_4 \cdot k_3$	$q_4 \cdot k_4$	
$q_5 \cdot k_1$	$q_5 \cdot k_2$	$q_5 \cdot k_3$	$q_5 \cdot k_4$	$q_5 \cdot k_5$

Matrix 2 (Fixed): A 5x5 matrix representing relative distances between positions. The value in row i and column j is $j - i$.

0				
-1	0			
-2	-1	0		
-3	-2	-1	0	
-4	-3	-2	-1	0

The two matrices are added together, and the result is scaled by m .

Маскирование

Маскирование в контексте трансформеров - это процесс скрывания информации о будущих словах в последовательности при обучении модели. Это означает, что при обучении модели на задаче предсказания следующего слова в последовательности, модель не имеет доступа к информации о будущих словах в последовательности.

Маскирование важно в трансформерах по нескольким причинам:

- Обучение на задаче предсказания следующего слова
- Учет контекста
- Универсальность

В трансформерах маскирование обычно реализуется с помощью механизма внимания, который позволяет модели фокусироваться на определенных частях входной последовательности при предсказании следующего слова. Маскирование также может быть реализовано с помощью специальных токенов, таких как [MASK], которые заменяют случайные слова в последовательности при обучении модели.

Обучение трансформеров с нуля

- Только кодировщик:
 - Классификация текстов
 - Сравнение текстов
 - Классификация слов в тексте
 - Отбор ответов на вопрос
 - ...
- Только декодировщик:
 - Генерация текста
 - Генерация ответов на задания

Обучение трансформеров с нуля

- Требуется огромного объёма данных
- Требуется колоссальных вычислительных мощностей
- Можно ли заранее обучить большую модель так, чтобы она легко донастраивалась на новые задачи?